

Исправления для сборки Webpack/Next.js

⚠ ЧАСТИЧНО ЗАВЕРШЕНО - ОСТАЛИСЬ ТИПЫ

Применённые изменения ☒

1. ☒ Убран fileLogger из `error.tsx` и `not-found.tsx`
2. ☒ Добавлен runtime: 'nodejs' для middleware
3. ☒ Добавлен outputFileTracingRoot в next.config.js
4. ☒ Улучшен `/api/error/route.ts`
5. ☒ Создан `api-error-handler.ts`
6. ☒ Добавлен ioredis для rate limiting
7. ☒ Убран fileLogger из `crypto.ts` → заменён на console

Остались проблемы **✗**

TypeScript ошибки в API routes:

- `import db from '@lib/database'` → нужно заменить на `import { getDatabase } from '@lib/database'`
- `const db = await getDatabase()` → нужно заменить на `const db: any = await getDatabase()`

Файлы для исправления:

```
src/app/api/admin/backup/route.ts
src/app/api/admin/backup/restore/route.ts (ЧАСТИЧНО ИСПРАВЛЕНО)
src/app/api/chats/[id]/pin/route.ts
src/app/api/chats/group/create/route.ts
src/app/api/chats/group/members/route.ts
src/app/api/installer/clear/route.ts
src/app/api/installer/test-accounts/route.ts
src/app/api/messages/route.ts
src/app/api/pages/route.ts
src/app/api/profile/avatar/upload/route.ts
src/app/api/profile/password/route.ts
src/app/api/statuses/route.ts
src/app/api/users/[id]/block/route.ts
src/app/api/yandex-disk/link/route.ts
```

Команда для массового исправления (PowerShell):

```
cd messenger
Get-ChildItem -Recurse -Path "src\app\api" -Filter "route.ts" | ForEach-Object {
    $content = Get-Content $_.FullName -Raw
```

```

    if ($content -match "import db from '@/lib/database'") {
        $content = $content -replace "import db from '@/lib/database'", "import {
getDatabase } from '@/lib/database'"
        $content = $content -replace "const db = await getDatabase\\(\\)", "const db:
any = await getDatabase()"
        Set-Content $_.FullName $content -NoNewline
        Write-Host "Fixed: $($_.FullName)"
    }
}

```

Остальная документация ...

Файл: `src/app/error.tsx`

Проблема: Клиентский компонент использовал `fileLogger`, который требует Node.js модули (`fs`, `path`).

Решение:

- Удалён импорт `fileLogger`
- Ошибки логируются в консоль клиента
- Данные об ошибках отправляются на `/api/error` для записи в файл на сервере

2. ☒ Добавлен runtime: 'nodejs' для middleware

Файл: `src/middleware.ts`

Проблема: Middleware использует Node.js-only модули (Redis), но по умолчанию Next.js пытается скомпилировать его для Edge Runtime.

Решение: Добавлена конфигурация:

```

export const config = {
  matcher: ['/api/:path*', '/admin/:path*', '/profile/:path*',
'/settings/:path*'],
  runtime: 'nodejs', // ← Добавлено
};

```

3. ☒ Добавлен outputFileTracingRoot в next.config.js

Файл: `next.config.js`

Проблема: Next.js не мог определить корень workspace из-за множественных lockfiles.

Решение: Добавлено:

```
outputFileTracingRoot: '/home/balloo/Messenger_Balloo_next_ts/',
```

4. ☒ Улучшен /api/error/route.ts

Файл: `src/app/api/error/route.ts`

Изменения:

- Добавлено поле `isServer` для различия клиентских/серверных ошибок
- Расширенное логирование (timestamp, userAgent, ip)
- Теперь API может принимать и логировать все типы ошибок

5. ☒ Создан api-error-handler.ts

Файл: `src/lib/api-error-handler.ts` (НОВЫЙ)

Назначение: Обёртка для автоматического логирования всех ошибок в API routes.

Пример использования:

```
import { withErrorLogging } from '@lib/api-error-handler';

export async function GET(request: NextRequest) {
  return withErrorLogging(request, async () => {
    // Ваш код API
    return NextResponse.json({ data: 'value' });
  }, 'GET /api/users');
}
```

6. ☒ Добавлен ioredis для rate limiting

Пакет: `iodis@5.10.1`

Назначение: Redis клиент для распределённого rate limiting в production.

Примечание: Если Redis не настроен, middleware автоматически переключается на in-memory storage.

Как использовать api-error-handler

Для простых GET/POST handlers:

```
import { NextRequest, NextResponse } from 'next/server';
import { withErrorLogging } from '@lib/api-error-handler';

export async function GET(request: NextRequest) {
  return withErrorLogging(request, async () => {
    const data = await fetchSomeData();
    return NextResponse.json(data);
  }, 'GET /api/users');
}

export async function POST(request: NextRequest) {
  return withErrorLogging(request, async () => {
    const body = await request.json();
    const result = await processData(body);
    return NextResponse.json(result);
  }, 'POST /api/users');
}
```

Для ручного логирования:

```
import { fileLogger } from '@lib/file-logger';

try {
  // ваш код
} catch (error: any) {
  fileLogger.error('[API /users] Error processing request', {
    error: error.message,
    stack: error.stack,
    userId: user?.id,
  });
  throw error;
}
```

Что проверить перед деплоем

- ☐ `.env.production` содержит все переменные
 - ☐ `REDIS_URL` установлен (опционально, для production rate limiting)
 - ☐ `JWT_SECRET` минимум 32 символа
 - ☐ `ENCRYPTION_KEY` минимум 32 символа
 - ☐ `VAPID_*` ключи сгенерированы
 - ☐ `SMTP_*` настроены для отправки email
-

Команды для деплоя

```
# 1. Обновить код
git pull origin main

# 2. Установить зависимости
npm install --omit=dev

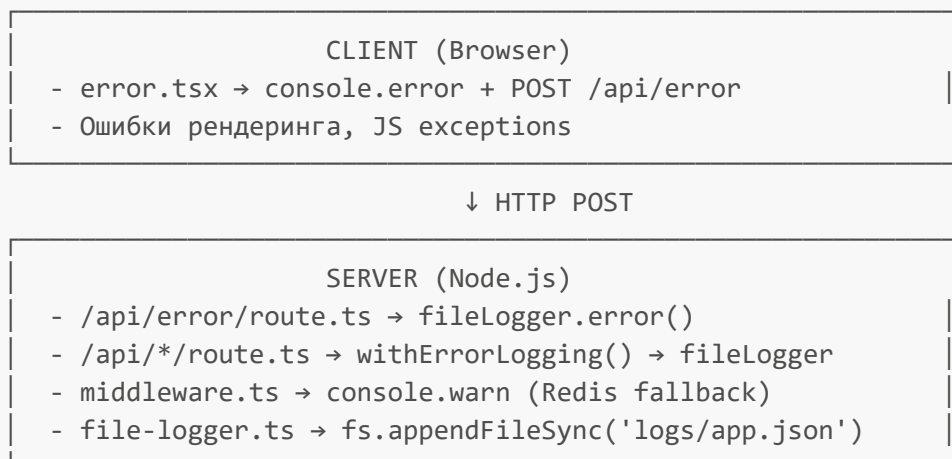
# 3. Удалить старый build
rm -rf .next

# 4. Собрать production build
NODE_ENV=production npx next build

# 5. Создать админа (если первый запуск)
npm run create-admin

# 6. Запустить
pm2 start npm --name "messenger-alpha" -- start
```

Архитектура логирования



Логи находятся в

- **Production:** `logs/app.json` (создаётся автоматически)
- **Формат:** JSON Lines (каждая строка - отдельный JSON объект)
- **Просмотр:** `tail -f logs/app.json` или через админку (если реализована)